

Claims Adjudication Workspace

A design for a claims platform that puts 20,000+ records and 1 GB documents in front of adjudicators without either one reaching the browser intact. This document covers the architecture, the performance strategy, and — most importantly — where each decision stops being the right one.

Scope Frontend architecture + BFF Stack Next.js 16 · React 19 · TypeScript

Ref 137263

→ [Working prototype: swiss-re-claims-workspace.vercel.app/claims](https://swiss-re-claims-workspace.vercel.app/claims)

Everything described here is implemented and running. Sign in as any of four roles; the access model, the range-streamed 2,400-page document and the cancellable split jobs are all live.

CONTEXT THE PROBLEM BENEATH THE REQUIREMENTS

Two hard problems wearing one application

ABC Insurance is replacing a legacy claims system. Claims arrive from mixed channels — email, SFTP drops, structured feeds — and adjudicators work them in a queue, then work *inside* the attached documents. That is two genuinely different engineering problems sharing a URL bar:

- **A wide-and-shallow problem.** 20,000+ claims, sorted and filtered many ways, where the user needs the right ten rows fast.
- **A narrow-and-deep problem.** One document of 100 MB–1.5 GB, where the user needs page 1,800 and nothing else — and then wants to split it.

They fail differently. The grid fails by re-rendering; the document fails by exhausting memory. A single strategy applied to both is how this kind of application ends up slow in two directions

at once, so this document treats them separately and only shares what genuinely deserves sharing.

ASSUMPTION ON RECORD

The brief states document sizes as **100 MB–1 GB** in the Context section and **1500 MB–1 GB** in the requirements. I have designed for the wider reading — **up to ~1.5 GB** — because every technique that survives 1.5 GB also survives 100 MB, and the reverse is not true.

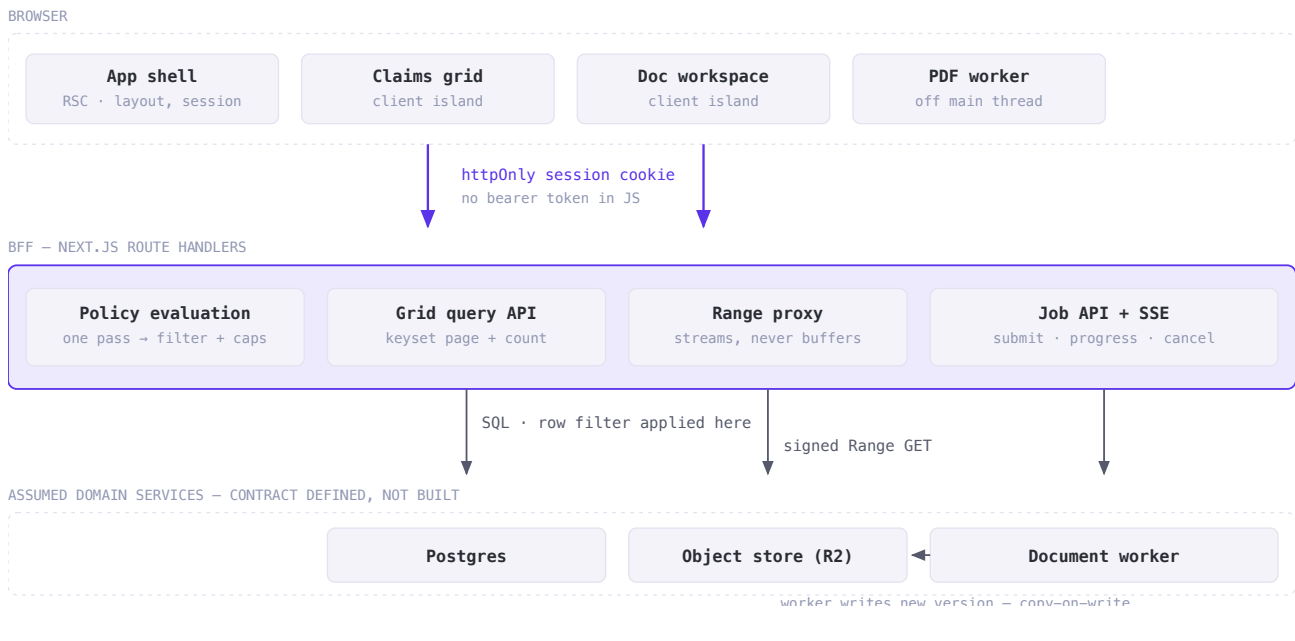
ARCH SYSTEM SHAPE

A thin BFF, and an honest boundary

The evaluation criteria ask for "*backend API assumptions*" — which tells me the domain services belong to another team. So this design does not build a backend; it defines the contract it consumes, and inserts one deliberate layer of its own: a **Backend-For-Frontend**.

The BFF exists for three reasons, and would be deleted if none of them held:

- The browser must never hold a downstream service token. The session is an `httpOnly` cookie; token exchange happens server-side.
- Role must be projected into *both* the data query and the capability flags the UI reads — from one policy evaluation, so they cannot drift apart.
- A 1.5 GB document needs a range proxy that streams and never buffers.



Every client request terminates at the BFF. The two client islands hold no credentials; the session cookie is the only thing that crosses. Policy is evaluated once per request and produces two outputs from the same decision — the SQL row filter and the capability flags the UI renders from — which is what stops the button state and the data scope from drifting apart.

Why Next.js, and what it is *not* doing

Next.js here is a host for the BFF, not an endorsement of rendering everything on the server. Route Handlers give real HTTP endpoints, middleware gives edge session checks, and `ReadableStream` responses give SSE for job progress — all in one deployable unit.

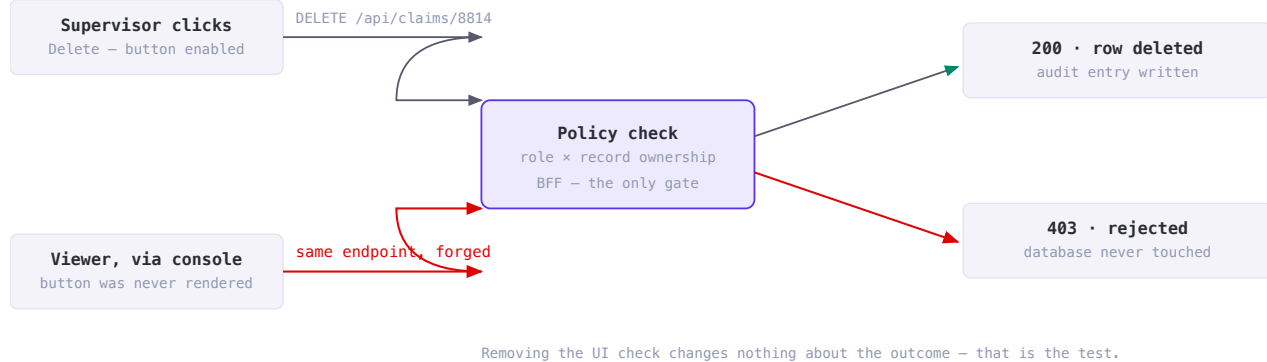
The grid is deliberately not a Server Component. Server-rendering a surface with client-side sort, filter and virtualised scroll turns every interaction into a network round-trip and a payload. RSC covers the shell, the layout and session bootstrap. Knowing where to stop using the framework's headline feature is the decision, not an omission.

FR-2 ROLE-BASED ACCESS

The hidden button is not the control

The brief is explicit: *backend as source of truth; frontend for UX*. The distinction matters because the two mechanisms look identical when everything works and only diverge under attack.

Each response carries a `capabilities` object alongside the data — derived from the same policy evaluation that built the row filter. The UI reads it to disable or hide actions. That is a *courtesy*: it stops adjudicators clicking things that will fail. It is not the boundary.



Both paths converge on one check. The forged request is indistinguishable from the legitimate one at the network layer, which is exactly why the frontend cannot be the gate. In the prototype this is demonstrable: disable the client-side capability check and the Viewer's delete still returns 403.

Row scope, not just action scope

RBAC here has two halves and only one of them is visible. **Action scope** controls Edit / Delete / Assign. **Row scope** controls which claims exist at all — an adjudicator sees their assigned queue, a supervisor sees their team's, an auditor sees everything read-only. Row scope is a `WHERE` clause added by the BFF and never sent by the client, so a tampered filter parameter can only ever narrow the set, never widen it.

FR-1 THE 20,000-ROW GRID

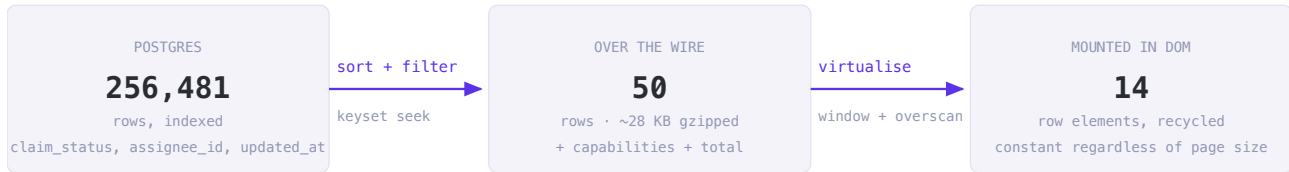
Pagination and virtualisation are not rivals

The brief asks us to use pagination *or justify an alternative*. The provided Figma settles it: the footer reads `Showing data 1 to 8 of 256K entries` with numbered pages. The client's own design standard specifies paginated navigation with a total count. Overriding that would be overruling their UX team, which is the wrong instinct for this role.

But the two techniques answer different questions, and treating them as alternatives is the actual mistake:

- **Pagination is a transport decision** — how many rows cross the network.
- **Virtualisation is a render decision** — how many rows exist in the DOM.

They compose. Here is what each hop actually removes:



Sorting and filtering never run in the browser — they run where the index is.
Scaling record count changes the left number only. The other two are already bounded.

The right-hand number is the one that protects the frame budget. It stays at 14 whether the page holds 50 rows or 500, and whether the table holds 20,000 claims or 2 million — which is what makes the design survive the growth the brief asks about.

The count problem the Figma creates

Numbered pagination requires a total. `COUNT(*)` across 256K filtered rows on every keystroke is the query that quietly takes the database down. And `OFFSET 320000` is fine at page 2 and pathological at page 6,400, because Postgres walks every skipped row.

So the count is tiered rather than uniform:

- **Up to 50,000 rows the count is exact.** At that size `COUNT(*)` against the filter index is single-digit milliseconds — cheap enough to run on every change, and worth running because the number is then unarguable.
- **Beyond it, the planner's row estimate takes over,** and the interface prefixes the number with `~` so an estimate is never passed off as exact.

The threshold started far lower and was raised after using it. Two problems surfaced, and both are worth stating because they are the kind that only appear in the running product: an estimate *does not move when a row is deleted* until the table is analysed, so the total contradicted the action the user had just taken; and an "approx." label beside a figure that happened to be exactly right read as though the underlying data were unreliable.

Both are failures of reaching for a scalability technique before the scale justified it. The technique is still correct at 256K rows — where the Figma's own footer places this screen — but correctness at the design's scale is not a reason to pay its cost at the current one.

Keeping re-renders honest

The grid uses no table library. With sorting, filtering and pagination all server-side, every row model such a library exists to provide is bypassed — what remains is a column config and a cell map, and owning those directly is what makes the render boundary controllable. A dependency you route around is a liability rather than an asset. The rules the grid holds to:

- Row identity comes from a stable server ID, never an array index, so recycling does not remount.
- Each row is memoised on the values that change its output, so scrolling past one row does not re-render the rows still on screen. No React Context is used anywhere near the grid — Context wakes every consumer on every change, which is the problem, not the fix.
- Filter input is debounced into the query key; the previous page stays mounted via placeholder data so pagination never flashes empty.
- Column definitions and cell renderers are declared outside the component so they are not reallocated per render.

DESIGN SYSTEM GAP

The Figma has **no row actions column** — but the brief requires Edit, Delete and Assign, each role-gated. We extend the system rather than improvise: an actions column following the existing 14px/#292D32 cell rhythm, a disabled state at 38% opacity matching the export's own `fill-opacity` convention, and a tooltip naming the missing permission. Flagged as a designer conversation, not resolved silently.

FR-3 OPENING A 1.5 GB DOCUMENT

The bytes that never move

This requirement has exactly one insight behind it, and everything else follows: **the document must never arrive in the browser**. A 1 GB `ArrayBuffer` exceeds what a tab can hold, and even if it fit, the user waited minutes to read one page.

So selecting a row does not fetch a document. It fetches a *manifest* — page count, per-page dimensions, byte offsets, thumbnail URLs — a few tens of kilobytes describing a 1.5 GB object. The workspace renders from the manifest immediately, then pulls only the byte ranges for pages actually on screen.

TOTAL TRANSFERRED TO OPEN THE DOCUMENT AT PAGE 1,800 — ABOUT 104 KB



The crossed-out arrow is the whole point. PDF.js background-fetches the entire file unless `disableAutoFetch` is set — a single option separating a working viewer from a dead tab. Range support requires `Accept-Ranges` and a CORS policy exposing `Content-Range`, which is why the object store must be one whose headers we control.

Getting to the workspace without a blank screen

Perceived performance is a stated requirement, so the transition is staged rather than awaited. Clicking a row does not block on anything:

1. The workspace opens immediately with claim metadata already held in the grid's cache — no spinner on data we have.
2. The manifest resolves and the page rail fills with skeleton tiles at correct aspect ratios, so the scrollbar is accurate before a single page has rendered.
3. Thumbnails stream in; the first full page rasterises.
4. The manifest is prefetched on row hover, so the common case has it before the click.

Keeping the main thread clear

Parsing runs in a Web Worker — non-negotiable, since PDF parsing on the main thread stalls scrolling on exactly the surface being scrolled. On top of that: at most five canvases are mounted with recycling, `renderTask.cancel()` fires when a page leaves the viewport, rendered pages are cached in IndexedDB keyed by document version and zoom bucket, and zoom is applied as a CSS transform first with re-rasterisation debounced behind it.

FR-4 DOCUMENT OPERATIONS

Annotations are cheap. Splits are not.

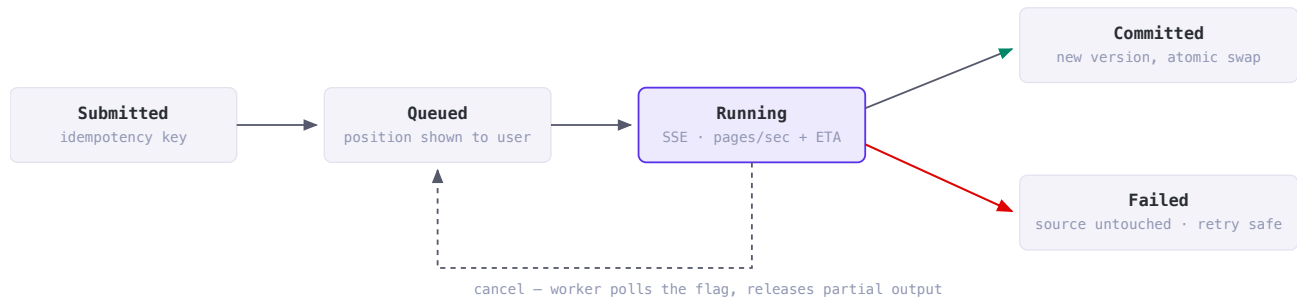
The brief groups edit, split, merge, delete, comments and annotations together. They should not be built together, because they differ on the only axis that matters: **whether the operation touches the document bytes**.

OPERATION	TOUCHES BYTES	EXECUTION	UPDATE MODEL	FAILURE COST
Comment on page	No	Row in Postgres	Optimistic	Toast + rollback
Annotation (highlight, box)	No	Row in Postgres	Optimistic	Toast + rollback
Reorder / delete page	Manifest only	Version bump	Optimistic, confirmed	Re-fetch manifest
Split document	Yes	Async worker job	Pessimistic	Job fails, source intact
Merge documents	Yes	Async worker job	Pessimistic	Job fails, sources intact

Annotations never touch the PDF. They are rows anchored to `(documentId, version, pageIndex, x, y, w, h)` in normalised coordinates, rendered as an overlay above the canvas. This buys three things: the document binary stays immutable, annotations survive a re-split by remapping page identity, and multiple adjudicators can annotate concurrently without fighting over a file. Burning them into the PDF happens once, at export, as its own job.

Split and merge as jobs, not requests

Splitting a 1.5 GB PDF takes far longer than any sane HTTP timeout. The client submits an *intent* — a page manifest, not bytes — and receives a job ID. Everything after that is progress reporting.



Retry re-sends the same idempotency key – a duplicate submission returns the original job, it does not split twice.

Copy-on-write is what makes partial failure survivable. The worker writes a new document version and swaps the pointer only on success, so a job that dies at 70% leaves the original exactly as it was. That is the brief's *"consistent document state"* and *"handle partial failures gracefully"* in one mechanism rather than two.

API CONTRACT WITH THE DOMAIN SERVICES

What the BFF assumes exists

GET	/api/claims cursor, limit, sort, filters → rows + capabilities + estimated total
PATCH	/api/claims/:id field-level edit; validated server-side against the same schema the form uses
POST	/api/claims/:id/assign reassign; rejected unless policy grants assign on that record
GET	/api/documents/:id/manifest page count, dimensions, byte offsets, thumbnail URLs, current version
GET	/api/documents/:id/stream

range proxy; forwards Range, returns 206, never buffers

GET	/api/documents/:id/annotations page-scoped, normalised coordinates
POST	/api/documents/:id/operations split / merge / delete intent + idempotency key → job ID
GET	/api/jobs/:id/events SSE stream — progress, ETA, terminal state
DELETE	/api/jobs/:id cooperative cancel

Request and response shapes are declared once as schemas and shared both ways — the BFF validates inbound payloads against them at runtime, and the client derives its types from the same definitions. Validation lives at the boundary, not in the form; the form's rules are a copy for responsiveness, and the server's copy is the one that counts.

TRADE-OFFS DECISIONS AND THEIR COSTS

What was chosen, and what it costs

Pagination **chosen** over infinite scroll **rejected**

- BECAUSE** The client's design standard specifies numbered pages and a total. Adjudicators also cite claim positions to each other — "page 4 of the unassigned queue" is a real conversation that infinite scroll destroys.
- COSTS** A total count must be maintained, which forces the estimated-count compromise above.
- REVISIT IF** The queue becomes a feed rather than a worklist, or mobile becomes primary.

Server-side sort and filter **chosen** over client-side **rejected**

- BECAUSE** 20,000 rows would fit in memory today — roughly 8 MB of JSON. 200,000 will not, and row-level RBAC cannot be enforced on a dataset the client already

holds.

COSTS

Every filter keystroke is a network round-trip; mitigated by debouncing and cached pages.

REVISIT IF

A view is provably bounded and non-sensitive — then a one-shot fetch is faster and simpler.

Optimistic annotations **chosen** · pessimistic splits **chosen**

BECAUSE

The correct model follows the cost of being wrong. A failed comment is a toast and a rollback. A half-applied split on a 1.5 GB legal document is a corrupted claim file.

COSTS

Two interaction models in one workspace, which must be taught through UI language — instant for annotations, an explicit progress panel for operations.

REVISIT IF

Never for splits. Destructive, expensive, irreversible operations do not get optimistic UI.

Server-side page rasterisation **chosen** over pure client parsing **rejected**

BECAUSE

Thumbnails for thousands of pages cannot be produced in the browser at acceptable cost, and rasterising once server-side serves every user who opens that document.

COSTS

Rasterisation infrastructure and a cache to maintain; first-open latency for a never-seen document.

REVISIT IF

Documents are small and rarely shared — then client-only rendering removes a whole service.

LIMITS WHERE THIS DESIGN ENDS

What I would change with more than a weekend

Stating the boundary is part of the design. These are the places where the prototype and the production architecture genuinely diverge:

- **The document worker cannot be serverless.** Splitting a 1.5 GB PDF exceeds serverless memory and timeout limits, and libraries that load the whole document into memory cannot be used at that size. Production needs a long-running container doing streaming page-range extraction, writing output via multipart upload. The prototype runs the real job protocol — idempotency, progress, cancel, copy-on-write — against documents small enough for the function to handle. The protocol is the part that has to be right; the executor swaps out.
- **Estimated counts should become a maintained aggregate.** A materialised count per common filter combination, refreshed by trigger, replaces the per-request estimate once query patterns are known from real usage.
- **Annotation concurrency is last-write-wins.** Adequate for one adjudicator per claim, which matches how the work is actually assigned. Genuine multi-user editing on the same page would need CRDTs or server-ordered operations, and I would want evidence that it happens before paying for it.
- **Policy is currently code.** Roles and capability rules live in a typed module in the BFF. At scale that belongs in a policy service so authorisation changes do not require a frontend deploy — but introducing that layer before the rules have stabilised makes them harder to read, not easier.
- **Accessibility is built in, not audited.** Semantic table markup, keyboard navigation through the virtualised grid, focus management on the grid-to-workspace transition, live-region announcements for job progress. A screen-reader pass on the annotation layer — the genuinely hard surface — is outstanding.

Prepared for the Swiss Re Senior Application Engineer (UI Developer) case study, reference 137263. Diagrams and the accompanying prototype apply the design tokens supplied in the CRM Dashboard Figma export.